

TITANIC SURVIVAL PREDICTION PROJECT REPORT

Name: Anamta Saleem

Domain : Data Science

Email Address: anamtakhadija2@gmail.com

Phone Number: 0336-3847369

Submitted To: Arch Technologies

Month Month 1

1. Introduction:

The Titanic dataset is a classic machine learning dataset used for classification problems. The goal of this project is to predict whether a passenger survived or not based on features such as age, gender, passenger class, fare, and family size.

This project involves data preprocessing, exploratory data analysis (EDA), feature engineering, and machine learning model training.

2. Data Preprocessing:

Steps performed:

- Handling missing values (Age, Embarked)
- Encoding categorical variables (Sex, Embarked)
- Feature engineering (FamilySize = SibSp + Parch + 1)
- Dropping irrelevant columns

3. Exploratory Data Analysis (EDA):

Key insights:

- Females had higher survival rates than males.
- 1st class passengers had better survival chances.
- Younger passengers had slightly higher survival probability.

Code and Explanations:

```
TITANIC SURVIVAL PREDICTION PROJECT :

# importing libraries:
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

[185] ✓ 0.0s

# importing Data
df = pd.read_csv('train.csv')
df.sample(5)

[186] ✓ 0.0s

'''
  PassengerId  Survived  Pclass
396         397         0       3
264         265         0       3
565         566         0       3
763         764         1       1
301         302         1       3
    Name      Sex  Age  SibSp  Parch  Ticket   Fare Cabin Embarked
396  Olsson, Miss. Elina  female  31.0    0    0   350407    7.8542   NaN      S
264  Henry, Miss. Delia  female   NaN    0    0   382649    7.7500   NaN      Q
565  Davies, Mr. Alfred J.  male   24.0    2    0   A/4 48871   24.1500   NaN      S
763  Carter, Mrs. William Ernest (Lucile Polk)  female  36.0    1    2   113760  120.0000  B96 B98      S
301  McCoy, Mr. Bernard  male   NaN    2    0   367226   23.2500   NaN      Q
'''
```

```

df.info()
[107] ✓ 0.0s

...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

```

df.columns
[112] ✓ 0.0s

...
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')

df.describe()
[113] ✓ 0.0s

...

```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```

df.duplicated().sum()
[114] ✓ 0.0s

...
np.int64(0)

df.isnull().sum()
[115] ✓ 0.0s

...
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64

```

In the given data we can clearly see that the "Cabin" attribute has 687 null values which is in large amount and the "Cabin" attribute is also seems to be unimportant. So, here we will drop this attribute and make the data more clean and stable.

```

df['Cabin'].sample(5)
[195] ✓ 0.0s

...
787    NaN
253    NaN
485    NaN
139    B86
878    NaN
Name: Cabin, dtype: object

df.drop('Cabin', axis=1, inplace=True)
[196] ✓ 0.0s

df.columns
[197] ✓ 0.0s

...
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Embarked'],
      dtype='object')

```

```

df['Age'] = df['Age'].fillna(df['Age'].mean())
[120] ✓ 0.0s

```

```

df['Age'].isnull().sum()
[121] ✓ 0.0s

```

```

... np.int64(0)

```

```

df['Embarked'].unique()
[122] ✓ 0.0s

```

```

... array(['S', 'C', 'Q', nan], dtype=object)

```

```

print((df['Embarked']=='S').sum())
print((df['Embarked']=='C').sum())
print((df['Embarked']=='Q').sum())
df['Embarked'].isnull().sum()
[123] ✓ 0.0s

```

```

...
644
168
77

```

```

mode_value = df['Embarked'].mode()[0]
mode_value
[124] ✓ 0.0s

```

```

... 'S'

```

```

df['Embarked'].fillna(mode_value, inplace=True)
[125] ✓ 0.0s

```

[C:\Users\Ananta](#) Saleem\AppData\Local\Temp\ipykernel_20036\3993600559.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves differently from the original. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value, inplace=True)' instead.

```

df['Embarked'].fillna(mode_value, inplace=True)

```

```

df['Embarked'].isna().sum()
[126] ✓ 0.0s

```

```

... np.int64(0)

```

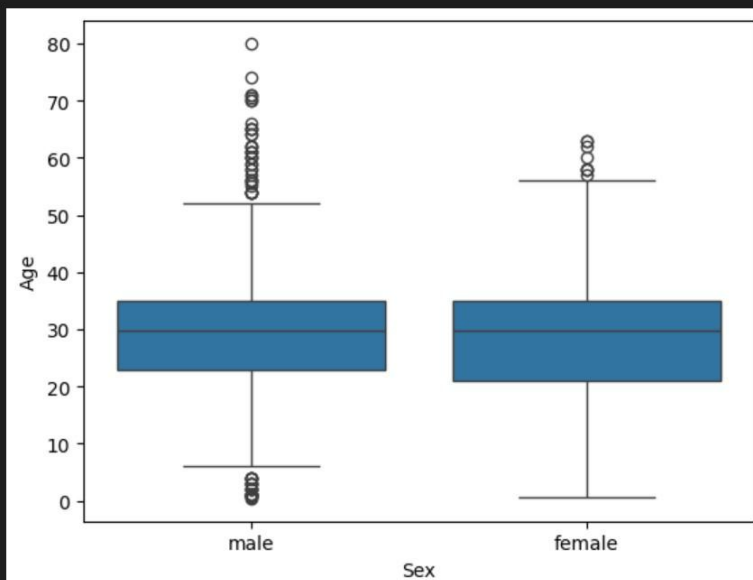
```
df.duplicated().sum()
[127] ✓ 0.0s
... np.int64(0)
```

Here we have cleaned our data from unnecessary missing and null values and make the data clean.

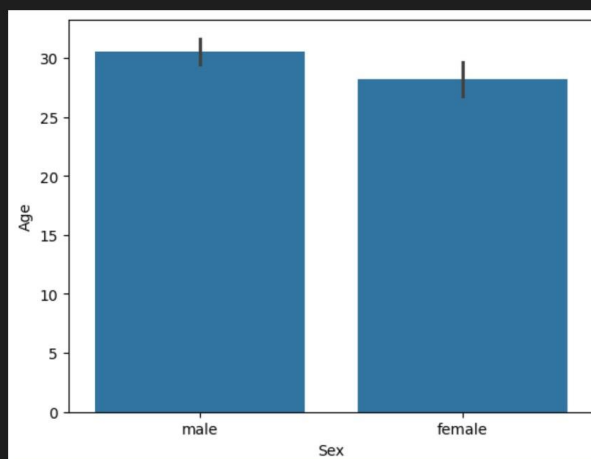
Finding Outliers

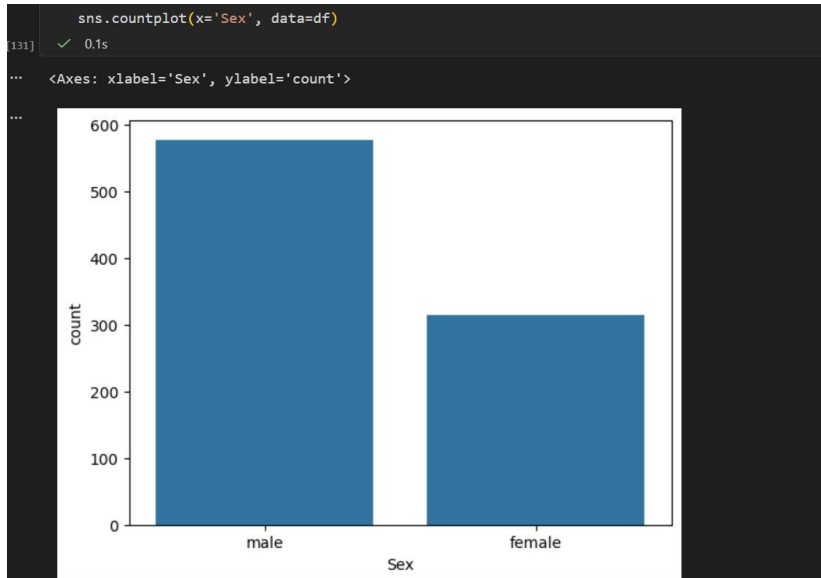
```
df.columns
[128] ✓ 0.0s
... Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
         'Parch', 'Ticket', 'Fare', 'Embarked'],
         dtype='object')
```

```
sns.boxplot(y='Age', x='Sex', data=df)
[129] ✓ 0.1s
... <Axes: xlabel='Sex', ylabel='Age'>
```



```
sns.barplot(x='Sex', y='Age', data=df)
[130] ✓ 0.2s
... <Axes: xlabel='Sex', ylabel='Age'>
```





```
Q1= df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("lower bound:", lower_bound)
print("upper bound:", upper_bound)
```

[132] ✓ 0.0s

[133] ✓ 0.0s

lower bound: 2.5
upper bound: 54.5

```
outliers = df[(df['Age'] < lower_bound) | (df['Age'] > upper_bound)]
print(outliers.shape)
```

[134] ✓ 0.0s

(66, 11)

```
outliers[['Age', 'Sex']].head(8)
```

[198] ✓ 0.0s

	Age	Sex
7	2.00	male
11	58.00	female
15	55.00	female
16	2.00	male
33	66.00	male
54	65.00	male
78	0.83	male
94	59.00	male

```
df['Age'].skew()
```

[136] ✓ 0.0s

np.float64(0.4344880940129925)

Outliers in Age are valid and represent real passengers, so they are not removed.

Insights of the data:

```
[137] ✓ 0.0s Python  
... df.columns  
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',  
      'Parch', 'Ticket', 'Fare', 'Embarked'],  
      dtype='object')
```

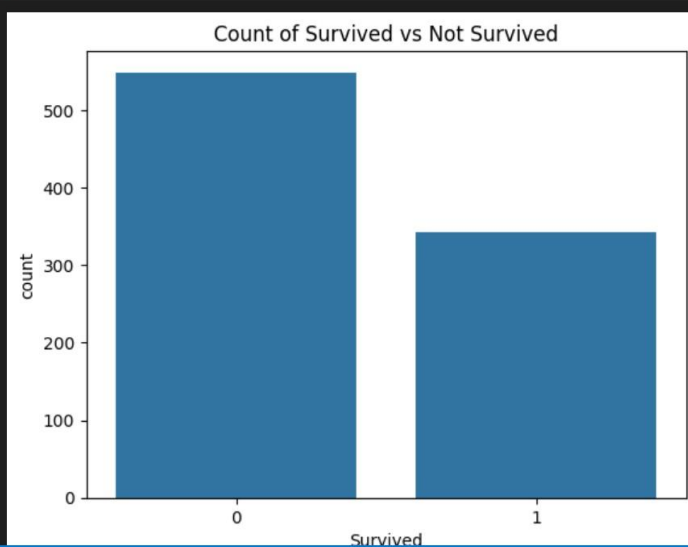
1. What is the overall survival distribution of passengers on the Titanic?

```
[199] ✓ 0.0s Python  
... df['Sex'].value_counts()  
Sex  
male      577  
female    314  
Name: count, dtype: int64
```

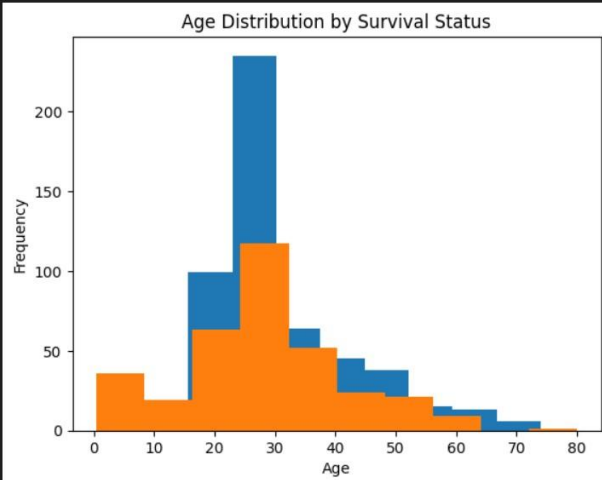
```
[138] ✓ 0.0s  
... df.groupby('Sex')['Survived'].mean()*100  
Sex  
female    74.203822  
male      18.890815  
Name: Survived, dtype: float64
```

"Females had higher survival (~70%) compared to males (~20%)"

```
[139] ✓ 0.1s  
... sns.countplot(x='Survived', data=df)  
plt.title('Count of Survived vs Not Survived')  
plt.show()
```



```
df.groupby('Survived')['Age'].plot(kind='hist')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution by Survival Status')
plt.show()
```



```
df['Survived'].mean() * 100
```

[141] ✓ 0.0s

```
np.float64(38.38383838383838)
```

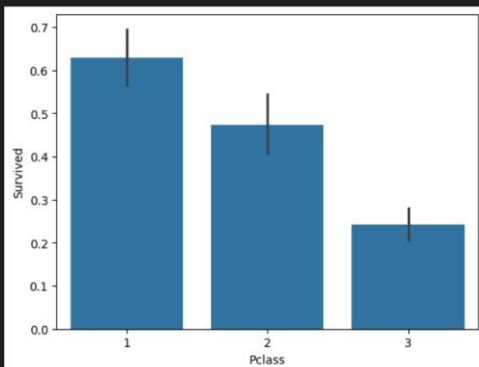
The number of passengers who did not survive is higher than those who survived.
The overall survival rate is around 38%, which shows that most passengers lost their lives.

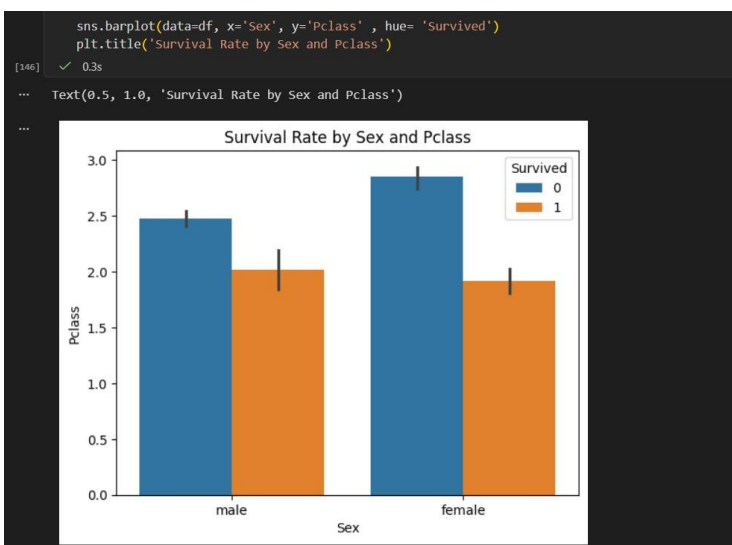
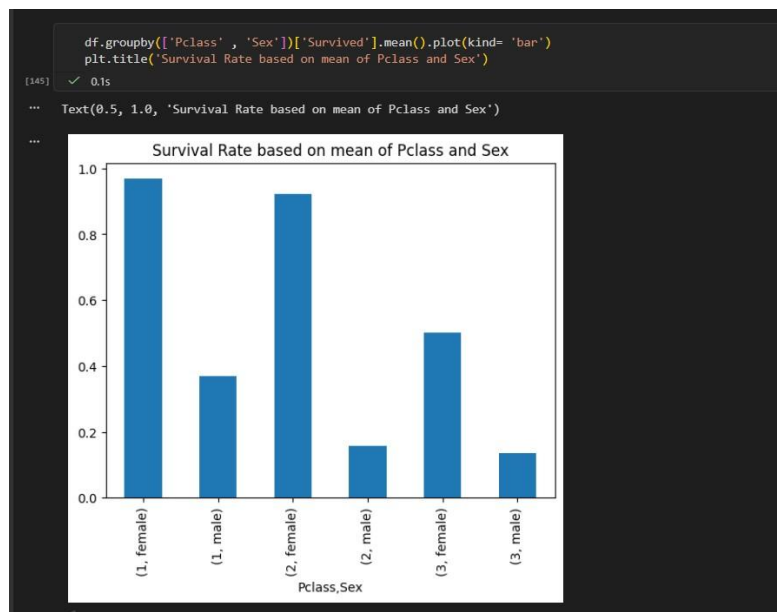
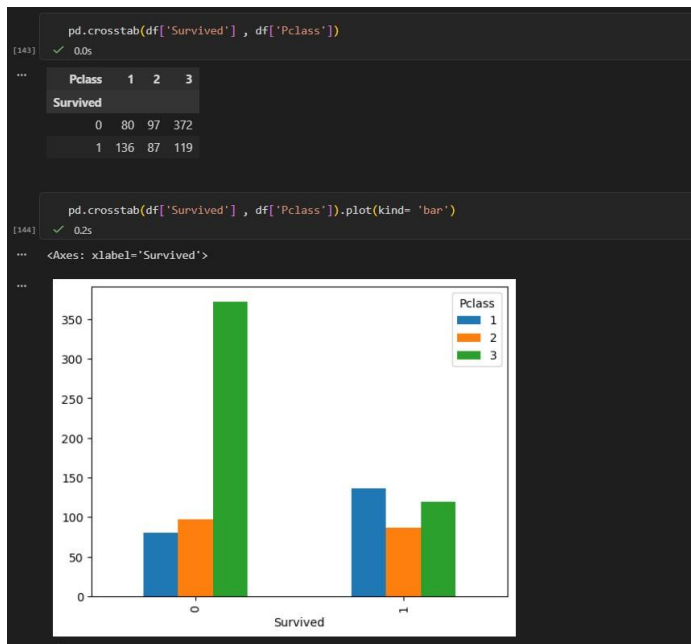
2. How many passenger survived based on the Class?

```
sns.barplot(y='Survived', x='Pclass', data=df)
```

[142] ✓ 0.4s

```
<Axes: xlabel='Pclass', ylabel='Survived'>
```





```
df.groupby(['Sex', 'Pclass'])['Survived'].value_counts().unstack()
```

```
[147] ✓ 0.0s
```

Sex	Pclass	Survived	0	1
female	1	3	91	
	2	6	70	
	3	72	72	
male	1	77	45	
	2	91	17	
	3	300	47	

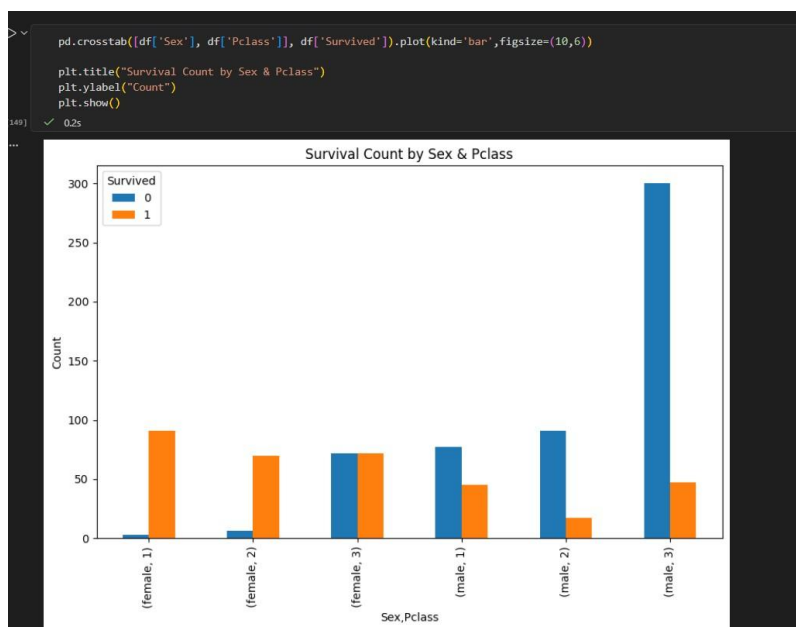
```
df.groupby(['Sex', 'Pclass'])['Survived'].mean()*100
```

```
[148] ✓ 0.0s
```

Sex	Pclass	Survived
female	1	96.808511
	2	92.105263
	3	50.000000
male	1	36.885246
	2	15.740741
	3	13.544669

Name: Survived, dtype: float64

Passengers in 1st class had the highest survival rate.
 Passengers in 3rd class had the lowest survival rate.
 This suggests that higher-class passengers had better access to safety measures.



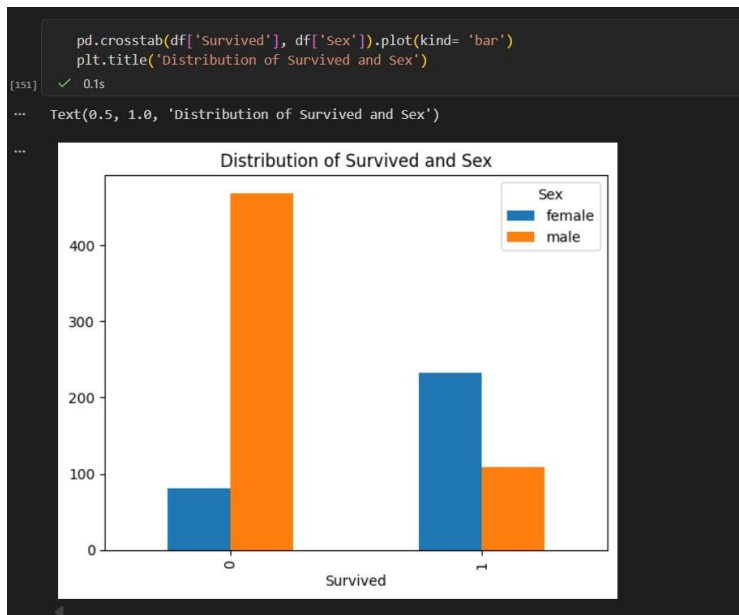
Females in 1st and 2nd class had the highest chances of survival.
 Males in 3rd class were the most affected group, with the lowest survival rate.
 This shows that both gender and class together strongly influenced survival.

3. How does survival vary between male and female?

```
pd.crosstab(df['Survived'], df['Sex'])
```

```
[150] ✓ 0.0s
```

Sex	female	male
Survived		
0	81	468
1	233	109



```
df.groupby('Sex')['Survived'].mean()
```

[152] ✓ 0.0s

... Sex
female 0.742038
male 0.188908
Name: Survived, dtype: float64

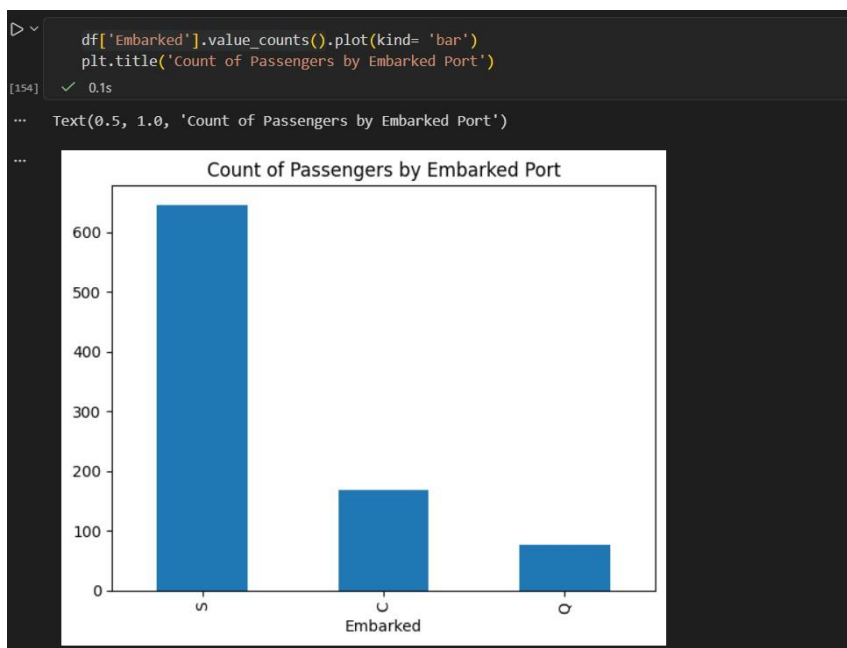
Females had a much higher survival rate compared to males.
Most of the males did not survive, while a large proportion of females were rescued.
This indicates a "women first" evacuation priority during the disaster.

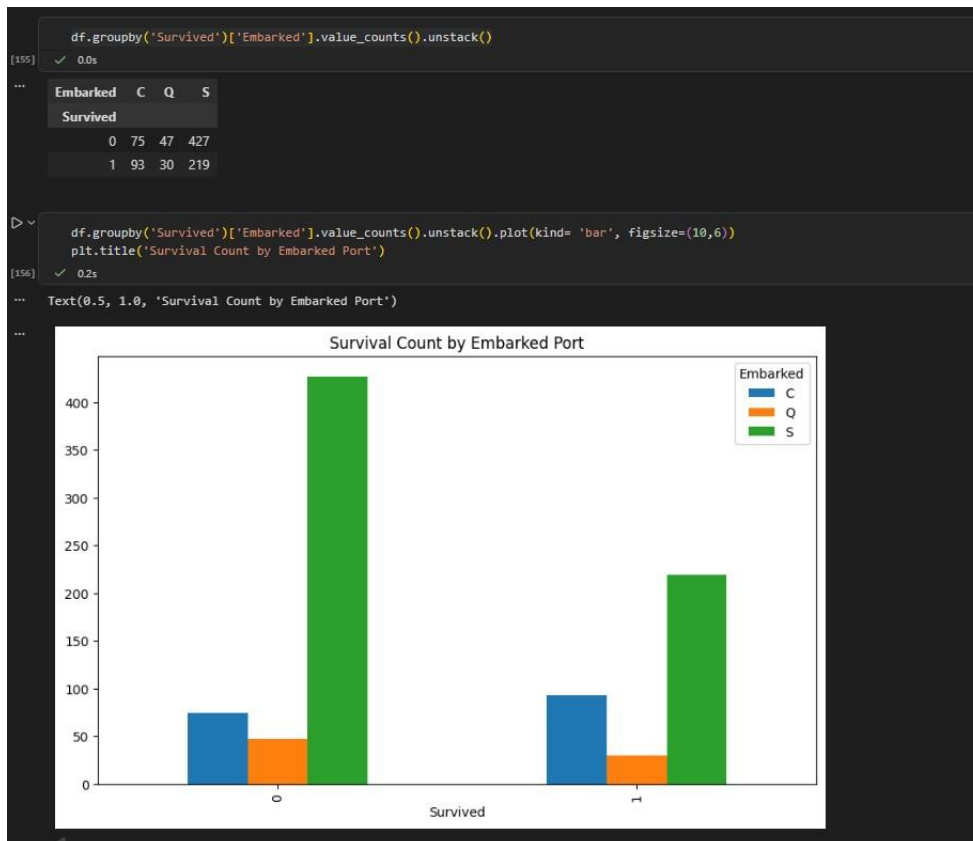
4. From which Embarked town most of the passengers had travel?

```
df['Embarked'].value_counts()
```

[153] ✓ 0.0s

... Embarked
S 646
C 168
Q 77
Name: count, dtype: int64

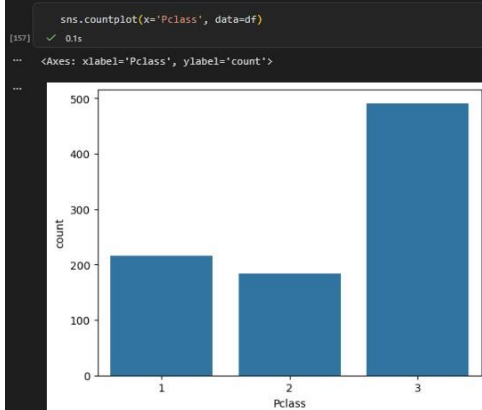


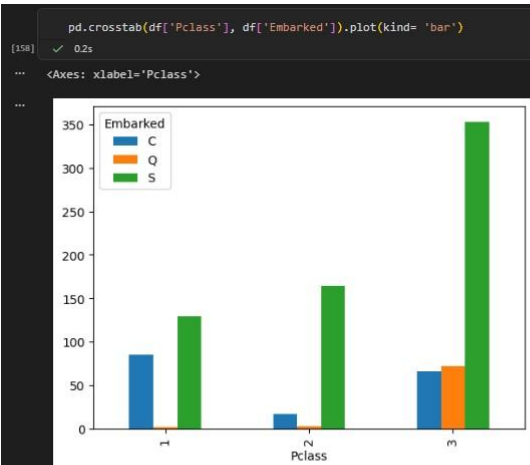


It is clear that most of the passengers were travelling from the "Southampton" about 646 passengers.
From "Cherbourg" there were 168 passengers were travelling.
From "Queenstown" there were 77 passengers.

Most passengers boarded from Southampton, and it also had the highest number of deaths.
Some ports showed slightly better survival rates, but this is mostly influenced by class distribution at each port.

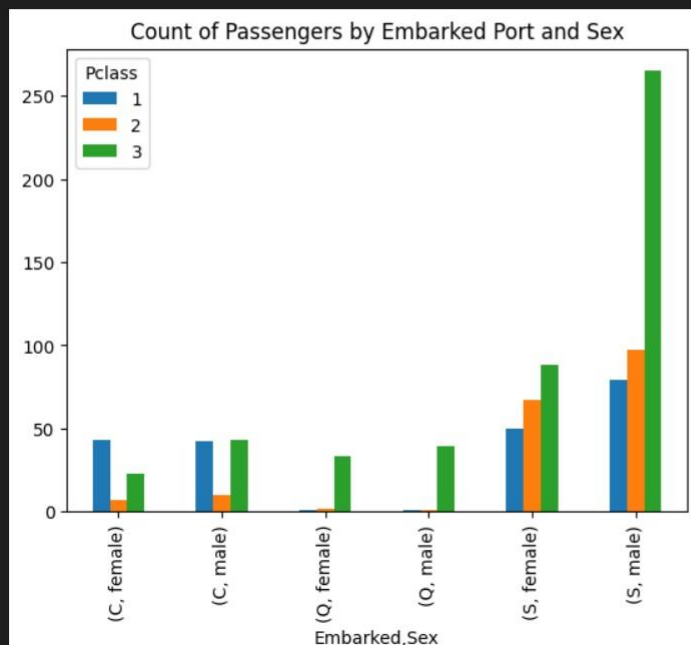
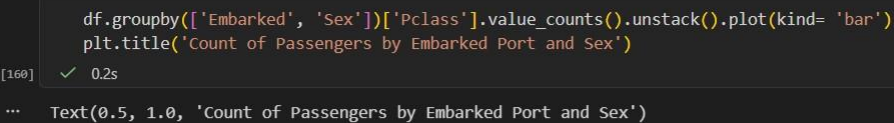
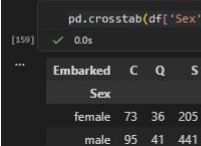
5. How many passengers were traveling from different towns based on Classes?

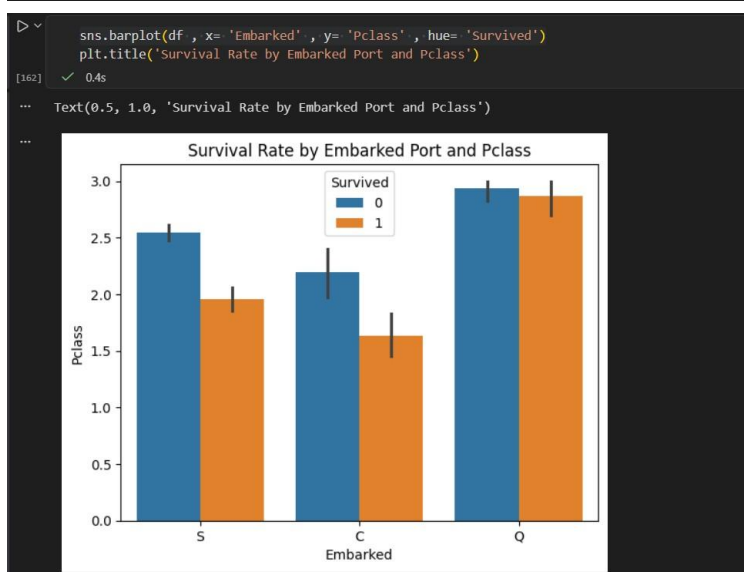
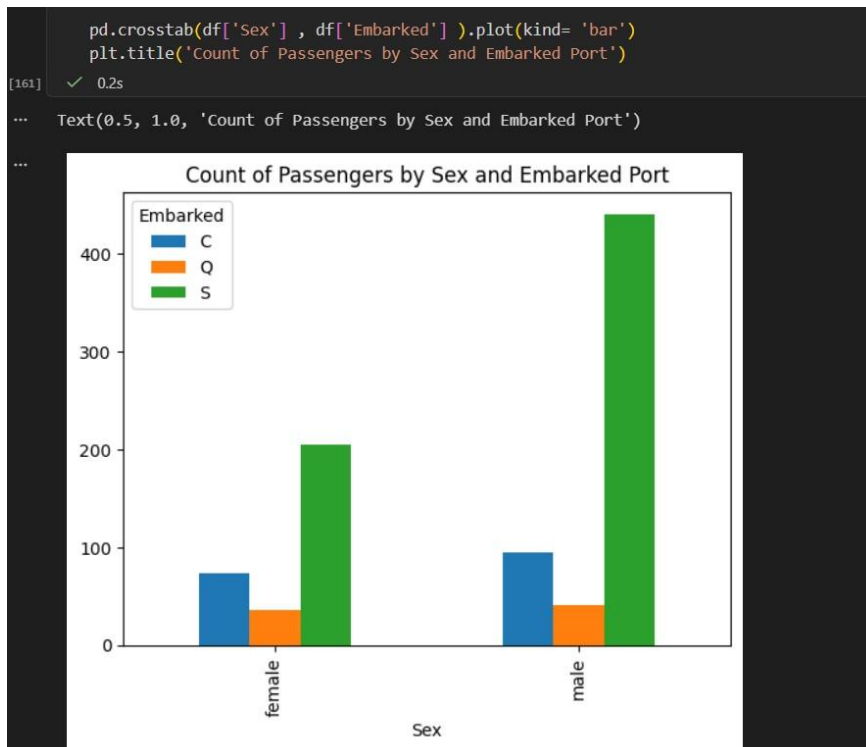




In class 1 : Southampton passengers = 129 | Cherbourg passengers = 85 | Queenstown = 2
 In class 2 : Southampton passengers = 164 | Cherbourg passengers = 17 | Queenstown = 3
 In class 3 : Southampton passengers = 353 | Cherbourg passengers = 72 | Queenstown = 66

6. How does gender distribution vary across different embarkation ports?





The analysis shows that female passengers had higher survival rates across all embarkation ports compared to males. Among the ports, Southampton had the highest number of passengers, and also the highest number of deaths, especially among males. Cherbourg showed relatively higher survival rates, particularly for female passengers, likely due to a higher proportion of first-class travelers. Male passengers across all ports, especially in lower classes, had significantly lower chances of survival.

7. How does fare vary across different embarkation ports (towns)?

```
df['Fare'].describe()
```

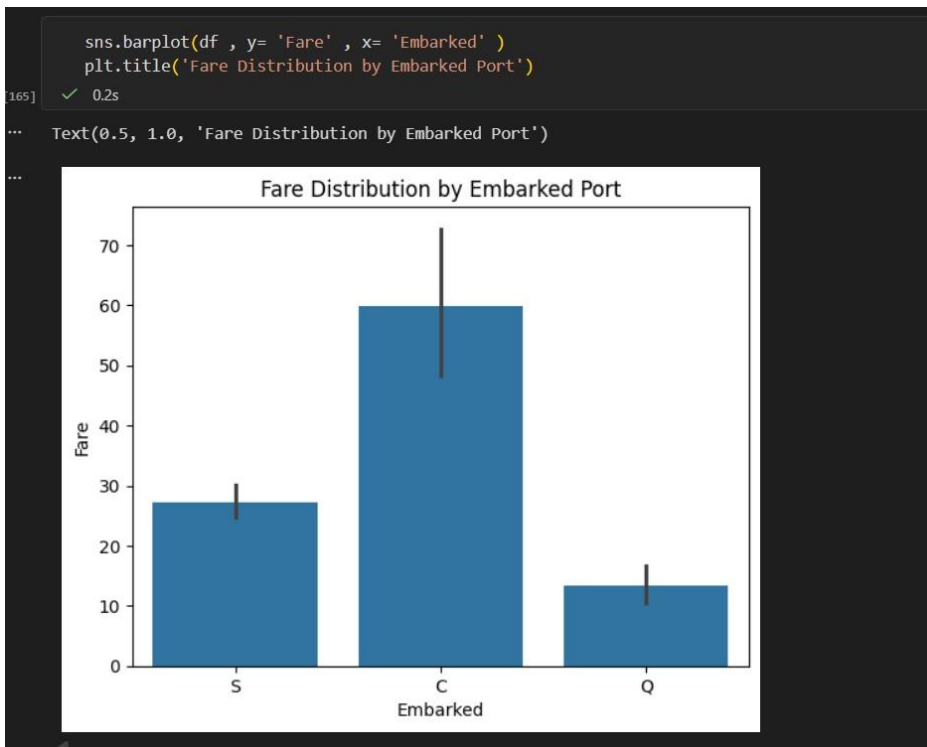
[163] ✓ 0.0s

```
df['Fare'].mean()
```

[164] ✓ 0.0s

np.float64(32.204287968574636)

Statistic	Value
count	891.000000
mean	32.204288
std	49.693429
min	0.000000
25%	7.910000
50%	14.454200
75%	31.000000
max	512.329200



```
df.groupby('Embarked')['Fare'].max()
```

[166] ✓ 0.0s

... Embarked

Embarked	Fare
C	512.3292
Q	90.0000
S	263.0000

Name: Fare, dtype: float64

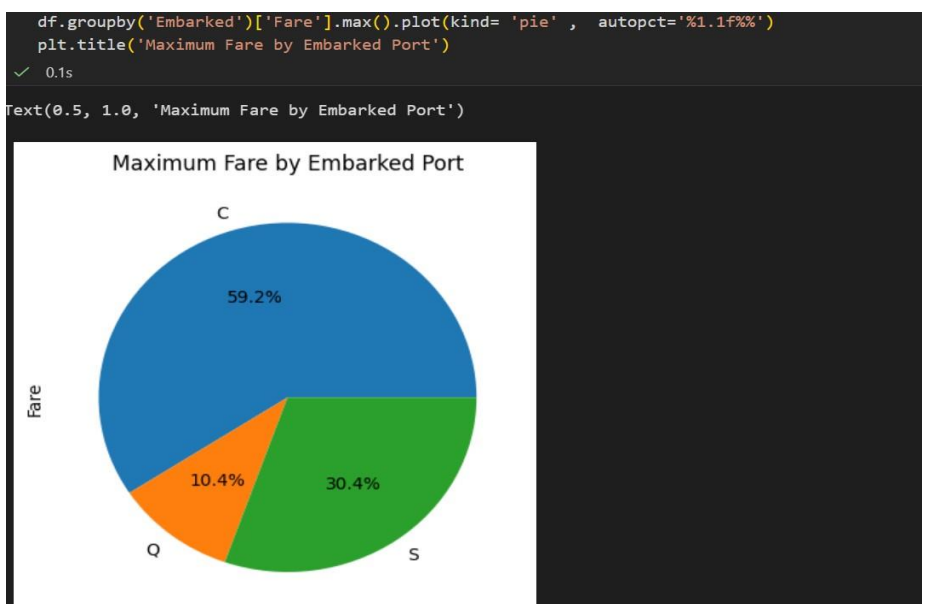
```
df.groupby('Embarked')['Fare'].min()
```

[167] ✓ 0.0s

... Embarked

Embarked	Fare
C	4.0125
Q	6.7500
S	0.0000

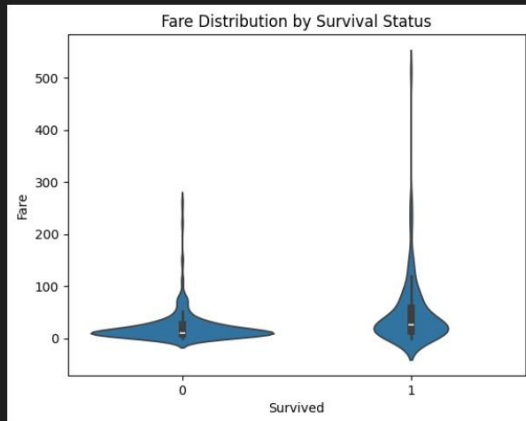
Name: Fare, dtype: float64



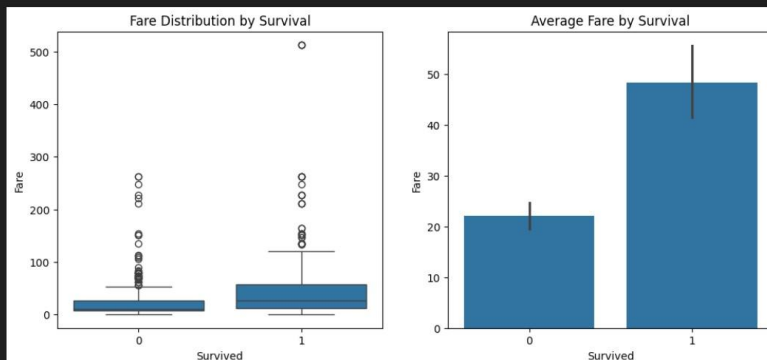
It is said that the "Cherbourg" has the highest fare among all the towns with 512 amount.

From C (Cherbourg) → highest fare = 512
From Q (Queenstown) → highest fare = 90
From S (Southampton) → highest fare = 263

```
sns.violinplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival Status')
[169] ✓ 0.2s
... Text(0.5, 1.0, 'Fare Distribution by Survival Status')
```



```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title("Fare Distribution by Survival")
plt.subplot(1,2,2)
sns.barplot(x='Survived', y='Fare', data=df)
plt.title("Average Fare by Survival")
plt.show()
[178] ✓ 0.3s
```



Passengers who paid higher fares had better survival chances.
This is because higher fare is linked with higher passenger class (Pclass).
So indirectly, wealth influenced survival probability.

8. What was the maximum and minimum age of the passengers?

```
df['Age'].min()
[171] ✓ 0.0s
... np.float64(0.42)

df['Age'].max()
[172] ✓ 0.0s
... np.float64(80.0)

df[df['Age'] <= 10]['Sex'].shape[0]
[173] ✓ 0.0s
... 64
```


9. What is the impact of family members (SibSp and Parch) on survival probability?

```
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df.columns

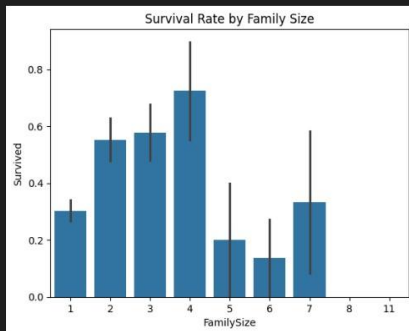
[174] ✓ 0.0s

Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Embarked', 'FamilySize'],
      dtype='object')
```

```
sns.barplot(x='FamilySize', y='Survived', data=df)
plt.title('Survival Rate by Family Size')

[175] ✓ 0.4s

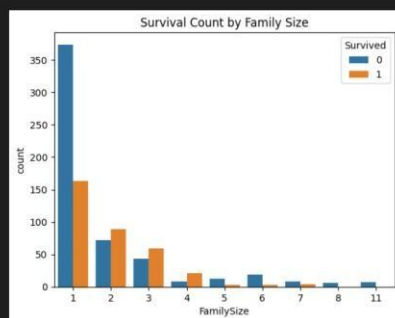
Text(0.5, 1.0, 'Survival Rate by Family Size')
```



```
sns.countplot(x='FamilySize', hue='Survived', data=df)
plt.title('Survival Count by Family Size')

[176] ✓ 0.3s

Text(0.5, 1.0, 'Survival Count by Family Size')
```

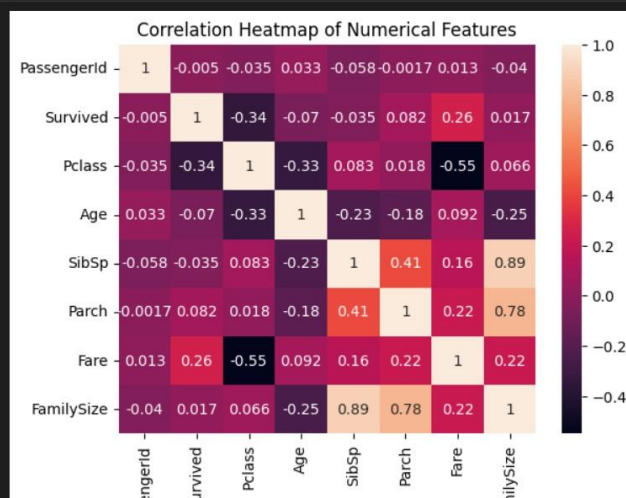


"Passengers with small family sizes (2-4 members) had the highest survival rate. Solo travelers and large families had lower survival probability. This suggests that being in a small group increased chances of survival due to better coordination and evacuation efficiency."

10. What is the relationship among Numeric Feature?

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()

[177] ✓ 0.6s
```



Survival is mainly affected by Pclass and Fare, showing that higher-class passengers had better survival chances. Fare and Pclass are strongly related, indicating socio-economic status played an important role. Age has very little impact, while Family features (SibSp, Parch) are strongly connected with each other.

Survival on the Titanic was primarily influenced by gender and socio-economic status, with females and higher-class passengers having significantly better chances of survival.

Model Development :

```
df.drop(columns=['PassengerId', 'Name', 'Ticket'], inplace=True)
```

[178] ✓ 0.0s

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.drop('Survived', axis=1), df['Survived'], test_size=0.2, random_state=42)
X_train.shape, X_test.shape
X_train.head()
```

[179] ✓ 0.0s

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FamilySize
331	1	male	45.5	0	0	28.5000	S	1
733	2	male	23.0	0	0	13.0000	S	1
382	3	male	32.0	0	0	7.9250	S	1
704	3	male	26.0	1	0	7.8542	S	2
813	3	female	6.0	4	2	31.2750	S	7

```
# One hot encoding
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
trf1 = ColumnTransformer([
    ('ohe_sex_embarked', OneHotEncoder(handle_unknown='ignore'), ['Sex', 'Embarked']),
], remainder='passthrough')
```

[180] ✓ 0.0s

```
#train the model
from sklearn.ensemble import RandomForestClassifier
trf2 = RandomForestClassifier(
    n_estimators=300,
    max_depth=6,
    min_samples_split=5,
    random_state=42
)
```

[181] ✓ 0.0s

Creating Pipeline:

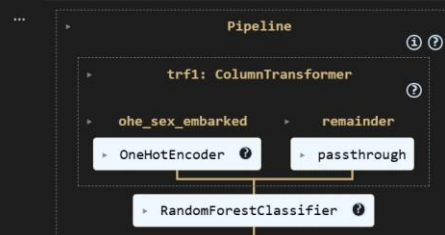
```
from sklearn.pipeline import Pipeline, make_pipeline

pipe = Pipeline([
    ('trf1', trf1),
    ('trf2', trf2),
])
```

[182] ✓ 0.0s

```
pipe.fit(X_train, y_train)
```

[183] ✓ 0.5s



```
y_pred = pipe.predict(X_test)
y_pred
[184] ✓ 0.0s
... array([0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0,
0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1,
0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0,
0, 1, 1])

# Accuracy of the model:
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy*100)
[185] ✓ 0.0s
... Accuracy: 81.00558659217877

from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X_train, y_train, cv=5)
print(scores.mean()*100)
[186] ✓ 2.7s
... 82.72136314389836
```

Testing the model:

```
import pandas as pd

sample = pd.DataFrame([{'Pclass': 1,
'Sex': 'male',
'Age': 80,
'SibSp': 0,
'Parch': 2,
'Fare': 20,
'FamilySize': 3,
'Embarked': 'Q'}])

prediction = pipe.predict(sample)[0]

if prediction == 1:
    print("The passenger will be: SURVIVED ")
else:
    print("The passenger will be: NOT SURVIVED")
[25] ✓ 0.0s
... The passenger will be: NOT SURVIVED
```

```
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(results.head(10))
[188] ✓ 0.0s
...


|   | Actual | Predicted |
|---|--------|-----------|
| 0 | 1      | 0         |
| 1 | 0      | 0         |
| 2 | 0      | 0         |
| 3 | 1      | 1         |
| 4 | 1      | 1         |
| 5 | 1      | 1         |
| 6 | 1      | 1         |
| 7 | 0      | 0         |
| 8 | 1      | 1         |
| 9 | 1      | 1         |


```

Conclusion:

In this project, we analyzed the Titanic dataset and performed data preprocessing, exploratory data analysis (EDA), and machine learning modeling to predict passenger survival. Important features such as passenger class (Pclass), gender (Sex), age, fare, and family size were used to build the model.

After training different models, the final pipeline achieved a reasonable accuracy in predicting survival outcomes. The model shows that gender, passenger class, and age are the most influential factors in survival prediction.

Overall, this project demonstrates how machine learning can be used to identify patterns in historical data and make predictions based on them.